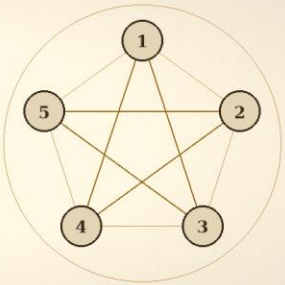
THE CURIOSITY SERIES · ARTICLE VI

Leslie Lamport

The man who taught computers how to agree.

Akshat Verma

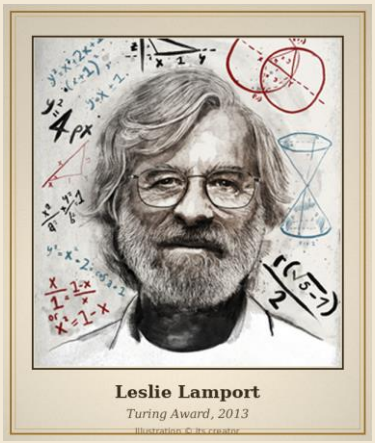
Distributed systems · consensus · formal methods



A TECHNICAL PROFILE · THE CURIOSITY SERIES, ARTICLE VI · AKSHAT VERMA

ABSTRACT

Distributed computing had no theory before Leslie Lamport gave it one. Between 1974 and 2002 he established how to order events without a shared clock, how many honest participants agreement requires when some participants lie, how unreliable machines can nevertheless choose a single value permanently, and how to state all of it precisely enough that a machine can check the reasoning. This paper walks through those four results, the arguments behind them, the eight years his most-used algorithm spent unpublished, and why an engineer working on AI systems in 2026 should care.



Leslie Lamport

Turing Award, 2013

Illustration © its creator

CONTENTS

1 No Shared Now	5 Consensus: Paxos	9 Why It Matters Now
2 Ordering by Causality	6 Specification, Not Testing	10 The Record
3 Agreement Amid Lies	7 The LaTeX Footnote	11 Conclusion
4 Replicated State	8 Where It Runs Today	12 References

1. NO SHARED "NOW"

Place two computers in different buildings and ask which of two events happened first. There is no honest answer. Quartz oscillators drift, network delay is unbounded and variable, and no observer anywhere sees a single global present. Before the late 1970s this was treated as an engineering nuisance to be papered over with tighter clock synchronisation and hope.

Lamport's first move was to recognise it as a conceptual problem rather than a hardware one, and then to solve it exactly by abandoning physical time altogether. The question "when did this happen?" has no reliable answer. The question "could this have caused that?" does. That substitution is the foundation everything else rests on.

2. ORDERING BY CAUSALITY

Each process keeps an integer counter and increments it on every event. Every message carries the sender's counter. On receipt, the receiving process sets its own counter to the larger of its current value and the value in the message, then adds one. The rule takes one line and gives a total order consistent with causality [1].

2.1 *What the relation does and does not promise*

The guarantee runs one way only. If event *a* could have influenced event *b*, then a's timestamp is smaller than b's. The converse is deliberately absent: a smaller timestamp does not prove influence, and two events with unrelated counters may be genuinely concurrent. Lamport was explicit that this is a feature. Concurrency is real, and a scheme that pretended to order concurrent events would be lying.

Vector clocks later closed the gap, giving each process an N-element vector so that incomparable vectors identify

concurrency exactly. The cost is O(N) space per message against Lamport's O(1). Both remain in use; the choice is a straightforward trade between fidelity and overhead.

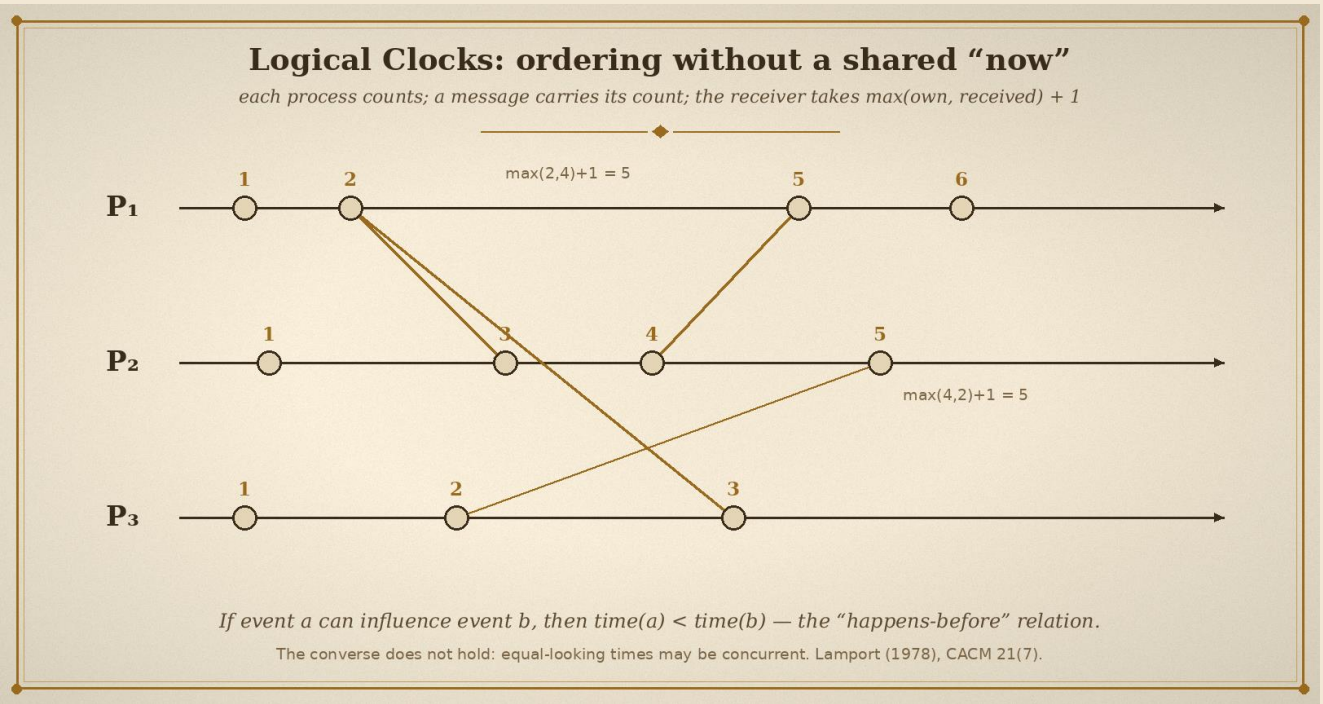


Figure 1 — Logical clocks. Three processes exchange messages; each receipt takes max(own, received) + 1.

3. AGREEMENT AMID LIES

Crash failures are the easy case: a component stops and everyone eventually notices. The hard case is a component that keeps running while reporting inconsistent information — telling one peer the attack is on and another that it is off. Faulty sensors, corrupted memory and compromised machines all behave this way.

With Marshall Pease and Robert Shostak, Lamport established the boundary precisely: to tolerate f arbitrarily faulty participants, at least **3f + 1** participants are required [2]. Three generals with one traitor is not a hard engineering problem; it is provably unsolvable. The result appeared first as a dry theorem, then two years later in the allegory that made it famous [3].

On storytelling. Lamport chose the Byzantine framing deliberately. He had watched Dijkstra's dining philosophers attract enormous attention and concluded that the surest way to get a problem read is to dress it as a story. The lesson worked here — and misfired badly a decade later.

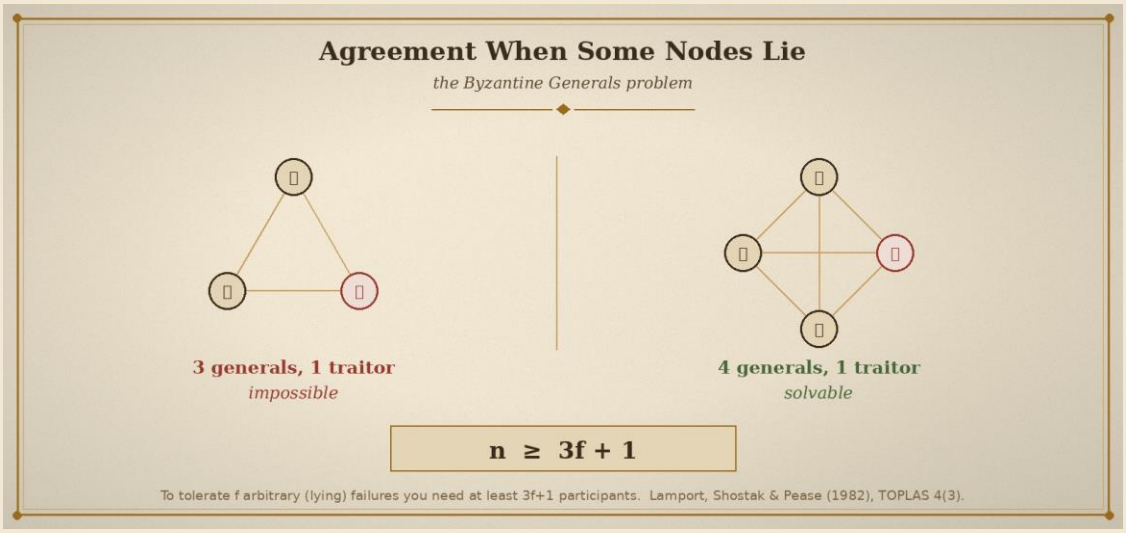


Figure 2 — Why 3f + 1. With three generals and one traitor no protocol succeeds; with four, agreement is achievable.

4. REPLICATED STATE

The practical goal behind the theory is the replicated state machine: run identical deterministic copies of a service on several machines, feed every copy the same sequence of commands, and any surviving copy can answer. Correctness reduces to one requirement — all replicas must agree on the order of commands. Lamport's ordering work supplies the sequence; the consensus work supplies the agreement.

5. CONSENSUS: PAXOS

Paxos answers the operational question directly. How does a group of unreliable machines settle on one value, permanently, when any of them may fail and messages may be lost, delayed or reordered?

5.1 *The protocol*

A proposer selects a proposal number and asks the acceptors to promise not to consider anything lower. If a majority promises, the proposer asks that majority to accept a value. Because any two majorities of a fixed group must share at least one member, no two conflicting values can both be chosen. Safety follows from the overlap, not from timing [4].

5.2 *What it cannot promise*

Safety survives crashes, delays and lost messages. Progress does not. Under full asynchrony the FLP impossibility result forbids any protocol from guaranteeing termination, so production deployments assume the network eventually behaves well enough for a stable leader to emerge. Systems that claim both properties unconditionally are mistaken about one of them.

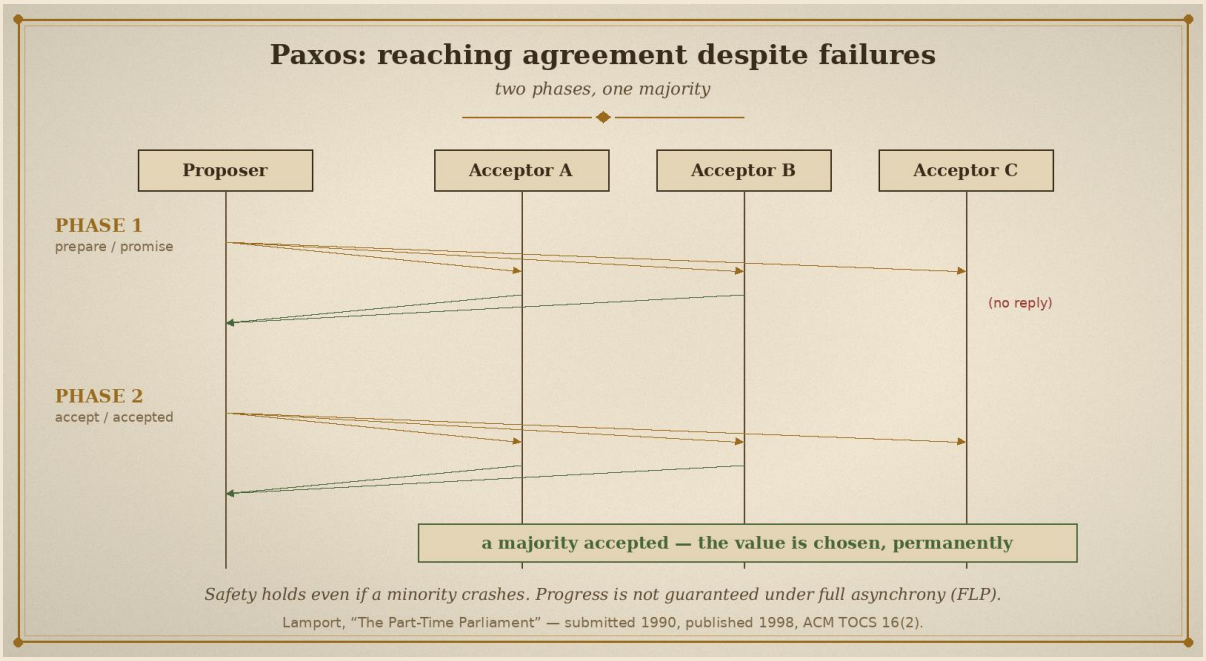


Figure 3 — Paxos. Two phases; a majority of acceptors makes the choice permanent. One acceptor may fail silently.

The eight-year detour. Lamport found Paxos while attempting to *disprove* a colleague's fault-tolerant file system — he set out to show the design could not work and discovered a working algorithm instead. He then wrote it up as an archaeological report on the lost parliament of a Greek island, complete with invented scholars. Submitted in 1990, it did not appear in print until 1998. His own assessment of the joke: "I created a story, and that was an utter disaster." He eventually relented and published a plain-language version [5].

6. SPECIFICATION, NOT TESTING

The thread joining all of it is a claim engineers still resist. You cannot test your way to correctness in a concurrent system, because the interleaving you never tried is the one that occurs at three in the morning under load. Testing samples the behaviour space; concurrency makes that space combinatorial.

Lamport's answer was the Temporal Logic of Actions and the TLA+ language built on it [6]: a notation for stating what a system must do, precisely enough that a model checker can explore the states for you. Amazon, Microsoft and Intel have used it to find design defects

before any code existed — the cheapest possible moment to find them.

7. THE LATEX FOOTNOTE

Along the way he wrote a macro package over Donald Knuth's TeX so that authors could describe document structure rather than typographic detail, and published the manual in 1986 [7]. It became the standard for scientific publishing and made him famous among millions of researchers who have never heard of Paxos. He has been mildly rueful about this ever since.

8. WHERE IT RUNS TODAY

Paxos and its descendants underpin Google's Chubby lock service and the systems built on it, Spanner's replication, and a broad class of configuration stores. Raft, published in 2014, deliberately traded elegance for teachability and now serves the same role in etcd, Consul and Kubernetes — an implicit compliment to how difficult the original was to read, and an explicit one to the problem Lamport framed.

9. WHY IT MATTERS NOW

We are again deploying systems whose behaviour cannot be exhaustively tested, only now they are probabilistic and far larger. The instinct is to rely on evaluation: run many cases, observe good outputs, ship. Lamport's career is a sustained argument that this is insufficient wherever failure is expensive, and that the alternative is to state properties precisely and prove things about them.

The analogy is imperfect and worth marking as such. Formal specification of a consensus protocol is tractable in a way that formal specification of a language model is not. What transfers is the discipline, not the tooling: decide what must never happen, write it down precisely, and treat confidence as no substitute for an argument.

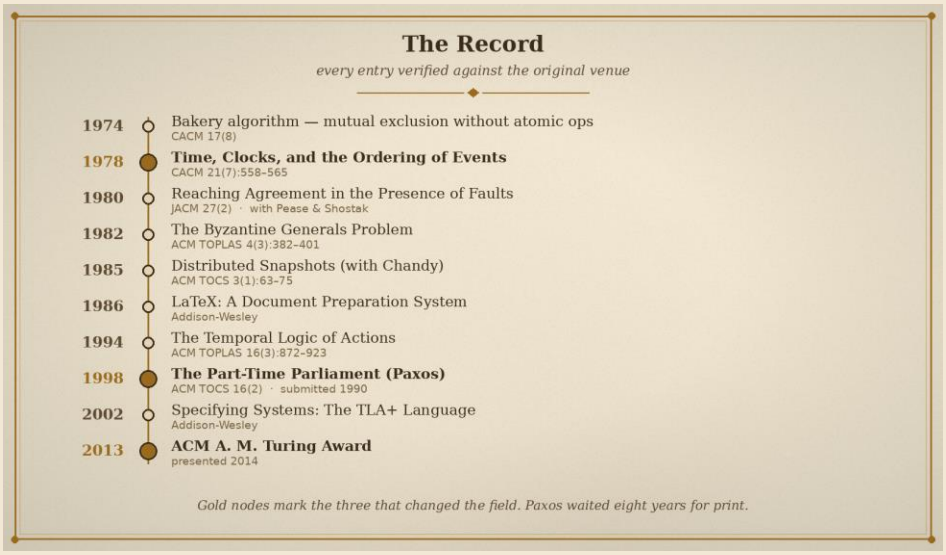


Figure 4 — The record. Every venue verified against the original publication.

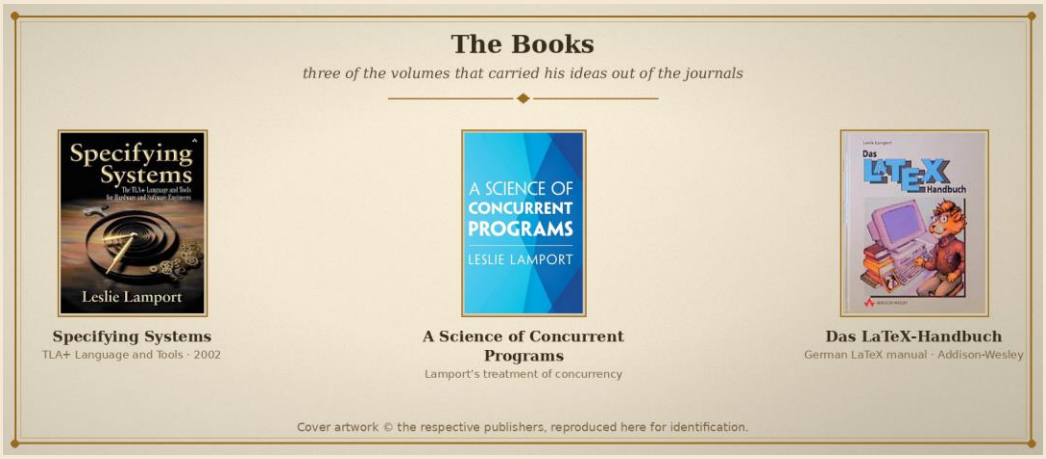


Figure 5 — Three of the books. Cover artwork © the respective publishers, shown for identification.

10. A NOTE ON SOURCES

Secondary summaries of Lamport circulate widely and are frequently wrong. In preparing this piece, one AI-generated source consulted placed the Byzantine Generals paper in the wrong journal, credited a three-author paper to Lamport alone, dated the Turing Award to 1997, and listed a Dijkstra Prize awarded eighteen years before the prize existed. Every citation below was checked against the publishing venue.

11. CONCLUSION

Lamport's subject was never really clocks, or generals, or parliaments. It was the question of what a distributed system can be *known* to do, as opposed to what it appears to do when observed. He supplied the vocabulary — happens-before, safety and liveness, replicated state machines, sequential consistency — that lets engineers argue precisely about systems nobody can watch in their entirety.

Asked what he was good at, his answer was not solving problems. "I was really good at asking questions" [8]. For a series built on the premise that the question is the hard part, there is no better place to end.

12. REFERENCES

[1] Lamport, L. "Time, Clocks, and the Ordering of Events in a Distributed System." CACM 21(7):558–565, 1978.

[2] Pease, M., Shostak, R. & Lamport, L. "Reaching Agreement in the Presence of Faults." JACM 27(2):228–234, 1980. Dijkstra Prize, 2005.

[3] Lamport, L., Shostak, R. & Pease, M. "The Byzantine Generals Problem." ACM TOPLAS 4(3):382–401, 1982.

[4] Lamport, L. "The Part-Time Parliament." ACM TOCS 16(2):133–169, 1998; submitted 1990.

[5] Lamport, L. "Paxos Made Simple." ACM SIGACT News 32(4), 2001.

[6] Lamport, L. "The Temporal Logic of Actions." ACM TOPLAS 16(3):872–923, 1994;

[7] Lamport, L. LaTeX: A Document Preparation System. Addison-Wesley, 1986.

[8] Microsoft Research, "Leslie Lamport Receives Turing Award," 2014. ACM A. M. Turing Award 2013, presented 2014.

[9] Chandy, K. M. & Lamport, L. "Distributed Snapshots." ACM TOCS 3(1):63–75, 1985.

[10] Lamport, L. Specifying Systems: The TLA+ Language and Tools. Addison-Wesley, 2002.

Next — Article VII.